

Project 2: Joint Detection of AI-Generated Images and Post-Processing Alterations in Real-World Scenarios

Master of Science in Artificial Intelligence

Students:

Paola Cortés Domínguez

Carmen Rojas Gallego

Course: Computer Vision

Professor: Irene Amerini

Date: 08/06/2026

Faculty of Information Engineering, Informatics and Statistics
Sapienza University of Rome

Contents

1	Introduction and Objectives	2
2	Data Management	2
3	Network Architecture	3
4	Training Strategy	4
5	Evaluation Methodology	6
6	Results	7
7	Discussion	8
8	Conclusions	8

1 Introduction and Objectives

The objective of this project is to train a convolutional neural network capable of analyzing images in real-world scenarios. Nowadays, images created by generative AI are mixed with real photos across different platforms. In this project, we work with images that can be original, downloaded from the internet, or re-digitized from a screen.

To solve this problem, we implement a Multi-Task Learning framework. Instead of using separate networks, our goal is to train a single model that can process any input image and simultaneously predict two things:

- Class: whether the image is real or an AI-generated Fake.
- Scenario: The specific post-processing it has undergone (Original, Internet/Transfer, or Screen Re-digitization).

2 Data Management

To prevent the model from collapsing due to the thousands of images contained in the original file, we selected a subset of only 300 photos from each category, resulting in a total of 1,800 images. This approach allows us to verify the functionality of our model using a perfectly balanced dataset.

This extraction and counting process is controlled by the following code configuration:

```
1 counts = {}
2 max_fotos = 300 # Subset selection per category
3
4 # Inside the tarfile loop:
5 key = f"{clase}_{esc}"
6 if counts.get(key, 0) < max_fotos:
7     tar.extract(member, path=LOCAL_DESTINATION)
8     counts[key] = counts.get(key, 0) + 1
```

Listing 1: Subset extraction and category filtering code.

These images are copied directly into a local folder within Google Collab so that the graphics card (GPU) can read them instantly during training, avoiding any download delays.

Before splitting the data, all images must be standardized so that the neural network can process them efficiently. We define a set of transformations to resize every image to 224×224 pixels, convert them into PyTorch tensors, and normalize them using standard ImageNet parameters.

```
1 data_transforms = transforms.Compose([
2     transforms.Resize((224, 224)),
3     transforms.ToTensor(),
4     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
5 ])
```

Listing 2: Data transformations.

Additionally, a custom dataset class ('RRDatasetLocal') is implemented to automatically scan the local directories and assign the correct ground-truth labels for both tasks (Class and Scenario) based on the folder structure.

```

1 class RRDatasetLocal(Dataset):
2     def __init__(self, root_dir, transform=None):
3         self.samples = []
4         self.transform = transform
5         for root, _, files in os.walk(root_dir):
6             for file in files:
7                 if file.lower().endswith(('png', 'jpg', 'jpeg')):
8                     p = os.path.join(root, file)
9                     p_parts = p.split('/')
10                    c = 1 if 'ai' in p_parts else 0
11                    if 'original' in p_parts: t = 0
12                    elif 'transfer' in p_parts or 'internet' in p_parts: t = 1
13                    else: t = 2
14                    self.samples.append((p, c, t))
15
16     def __len__(self): return len(self.samples)
17     def __getitem__(self, idx):
18         path, c, t = self.samples[idx]
19         img = Image.open(path).convert('RGB')
20         if self.transform: img = self.transform(img)
21         return img, c, t

```

Listing 3: Custom local dataset class.

To ensure the AI does not simplify memorize the photos and is put to a real test, we train the model using 80% of the extracted images (1,440). The remaining 20% (360 images) is reserved exclusively to test the performance of the trained model, ensuring it has genuinely learned to generalize rather than just learning the other images by heart. This division is performed automatically using the following code:

```

1 complete_dataset = RRDatasetLocal(LOCAL_DESTINATION, transform=data_transforms)
2
3 # 80% train y 20% validation
4 train_size = int(0.8 * len(complete_dataset))
5 val_size = len(complete_dataset) - train_size
6
7 train_dataset, val_dataset = random_split(complete_dataset, [train_size, val_size])
8
9 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
10 val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

```

Listing 4: Dataset Splitting and Data Loader Configuration.

3 Network Architecture

To implement the Multi-Task Learning framework, we adopt a hard parameter-sharing strategy using a pre-trained ResNet18 as the backbone network. This backbone acts as a shared feature extractor, leveraging knowledge from the ImageNet dataset to capture robust spatial structures, textures, and edges common to both forensic tasks.

```

1 import torchvision.models as models
2
3 class MultiTaskModel(nn.Module):
4     def __init__(self):
5         super(MultiTaskModel, self).__init__()
6         # We use pre-trained ResNet18 as a base (the "body" that recognizes shapes)
7         self.base_model = models.resnet18(weights='IMAGENET1K_V1')

```

The original fully connected layer of the ResNet18, designed for standard 1000-class classification, is removed and replaced with an `nn.Identity()` layer. This modification allows us to isolate the raw feature vector consisting of 512 dimensions (`num_fts`).

```

1      # We save the number of input features of the last layer
2      num_ftrs = self.base_model.fc.in_features
3
4      # We remove the original ResNet final layer
5      self.base_model.fc = nn.Identity()

```

Following this backbone, we append two independent, parallel linear classification heads that process the extracted features simultaneously during the forward pass:

- **Class Head (`fc_class`):** A linear layer that maps the 512 features to 2 output logits, corresponding to the binary authenticity classification task (0: Real, 1: AI-Generated Fake).
- **Scenario Head (`fc_trans`):** A linear layer that maps the same 512 features to 3 output logits, corresponding to the multi-class post-processing scenario detection (0: Original, 1: Internet/Transfer, 2: Screen Re-digitization).

```

1      # We create the TWO exit "heads":
2      # 1. Head for Class (Real vs Fake) -> 2 outputs
3      self.fc_class = nn.Linear(num_ftrs, 2)
4
5      # 2. Stage Head (Original, Transfer, Redigital) -> 3 outputs
6      self.fc_trans = nn.Linear(num_ftrs, 3)

```

Finally, the execution of the dynamic multi-task prediction process across both branches during the forward propagation is defined as follows:

```

1      def forward(self, x):
2          # The image passes through the body of the network
3          x = self.base_model(x)
4
5          # The result is divided into the two tasks
6          out_class = self.fc_class(x)
7          out_trans = self.fc_trans(x)
8
9          return out_class, out_trans

```

This setup allows the network to evaluate both tasks simultaneously with a single forward pass, optimizing computational efficiency during training and inference.

4 Training Strategy

The model was trained using the Adam optimizer with a learning rate of 10^{-4} and a batch size of 32 images. Training was performed for 10 epochs using the GPU available in Google Colab.

Since the model simultaneously solves two classification problems, two independent Cross-Entropy losses were defined:

- L_{class} for Real/Fake classification.
- L_{trans} for Transformation classification.

```

1      import torch
2      import torch.nn as nn
3
4      criterion_class = nn.CrossEntropyLoss()
5      criterion_trans = nn.CrossEntropyLoss()
6      optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
7      EPOCHS = 10

```

The final optimization objective corresponds to the weighted sum of both losses:

$$L_{total} = \lambda_{class}L_{class} + \lambda_{trans}L_{trans} \quad (1)$$

where λ_{class} and λ_{trans} control the relative importance of each task. For the experiments reported in this work, equal weights were assigned:

$$\lambda_{class} = 0.5, \quad \lambda_{trans} = 0.5 \quad (2)$$

This configuration gives identical importance to both objectives and represents the base-line multi-task setting. Inside the training loop, the forward pass, joint loss computation, and backpropagation are executed as follows:

```

1  LAMBDA_CLASS = 0.5
2  LAMBDA_TRANS = 0.5
3
4  for epoch in range(EPOCHS):
5      model.train()
6      running_train_loss = 0.0
7      for images, labels_class, labels_trans in train_loader:
8          images, labels_class, labels_trans = images.to(device), labels_class.to(device), labels_trans.to(device)
9
10         optimizer.zero_grad()
11         outputs_class, outputs_trans = model(images)
12
13         loss_class = criterion_class(outputs_class, labels_class)
14         loss_trans = criterion_trans(outputs_trans, labels_trans)
15         total_loss = (LAMBDA_CLASS * loss_class) + (LAMBDA_TRANS * loss_trans)
16
17         total_loss.backward()
18         optimizer.step()
19         running_train_loss += total_loss.item()

```

At the end of every epoch, the model was evaluated on the validation set in order to monitor generalization performance and detect potential overfitting. This real-time tracking is executed within a strict context without gradient computation:

```

1  # Real-time validation (strict exam)
2  model.eval()
3  running_val_loss = 0.0
4  with torch.no_grad():
5      for images, labels_class, labels_trans in val_loader:
6          images, labels_class, labels_trans = images.to(device), labels_class.to(device), labels_trans.to(device)
7          outputs_class, outputs_trans = model(images)
8
9          v_loss_class = criterion_class(outputs_class, labels_class)
10         v_loss_trans = criterion_trans(outputs_trans, labels_trans)
11         running_val_loss += (LAMBDA_CLASS * v_loss_class) + (LAMBDA_TRANS * v_loss_trans)

```

5 Evaluation Methodology

The evaluation phase was conducted exclusively on the validation subset composed of previously unseen images. For the Real/Fake classification task, the following metrics were computed:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

The confusion matrix provides a visual interpretation of the detector's behaviour by highlighting the number of correctly classified images and the most common classification errors. The classification report generated using Scikit-Learn summarizes the overall performance of the model and allows a direct comparison between the Real and AI-generated classes.

In addition, several randomly selected validation images were visualized together with the corresponding predictions to qualitatively inspect the model behaviour. To set up this inspection pipeline, the visualization grid and classification mappings are initialized as follows:

```
1 def test_and_display_random(model, dataset, n_images=5):
2     model.eval()
3     fig, axes = plt.subplots(1, n_images, figsize=(22, 6))
4
5     dic_class = {0: "REAL", 1: "AI (FAKE)"}
6     dic_trans = {0: "Original", 1: "Internet/Transfer", 2: "Pantalla (Redigital)"}
7
8     # We choose 'n_images' (5) indexes completely at random without repeating
9     indices = random.sample(range(len(dataset)), n_images)
```

Then, for each randomly extracted sample, a forward pass is executed inside a non-gradient context. The input image is reverted to its original color space by swapping the tensor dimensions and reversing the ImageNet standardization parameters before displaying:

```
1     for idx, i in enumerate(indices):
2         img, label_class, label_trans = dataset[i]
3
4         with torch.no_grad():
5             output_class, output_trans = model(img.unsqueeze(0).to(device))
6
7             pred_class = torch.argmax(output_class, dim=1).item()
8             pred_trans = torch.argmax(output_trans, dim=1).item()
9
10        #We undo the normalization to paint the photo properly
11        img_display = img.permute(1, 2, 0).cpu().numpy()
12        img_display = img_display * [0.229, 0.224, 0.225] + [0.485, 0.456, 0.406]
13        img_display = np.clip(img_display, 0, 1)
14
15        axes[idx].imshow(img_display)
```

Finally, text labels are retrieved via dictionary lookups, and a binary color-coding scheme is applied directly to the bounding titles to immediately highlight correct matches in green or misclassifications in red:

```

1     txt_real_class = dic_class[label_class]
2     txt_pred_class = dic_class[pred_class]
3     txt_real_trans = dic_trans[label_trans]
4     txt_pred_trans = dic_trans[pred_trans]
5
6     # Green if everything is correct, red if something is wrong
7     color_title = 'green' if (label_class == pred_class and label_trans == pred_trans) else 'red'
8
9     title = f"CLASE:\nReal: {txt_real_class}\nPred: {txt_pred_class}\n\n"
10    title += f"ESCENARIO:\nReal: {txt_real_trans}\nPred: {txt_pred_trans}"
11
12    axes[idx].set_title(title, color=color_title, fontsize=10, loc='left')
13    axes[idx].axis('off')
14
15    plt.tight_layout()
16    plt.show()

```

6 Results

After training the multi-task model on the selected subset of the RRDataset, the final evaluation was performed on the validation set containing 360 previously unseen images.

For the Real/Fake classification task, the model achieved an overall accuracy of **88%**. The detailed classification report is summarized in Table 1.

Class	Precision	Recall	F1-Score	Support
Real	0.85	0.89	0.87	165
AI (Fake)	0.90	0.87	0.88	195
Accuracy	0.88			
Macro Average	0.88	0.88	0.88	360
Weighted Average	0.88	0.88	0.88	360

Table 1: Classification report for the Real/Fake detection task.

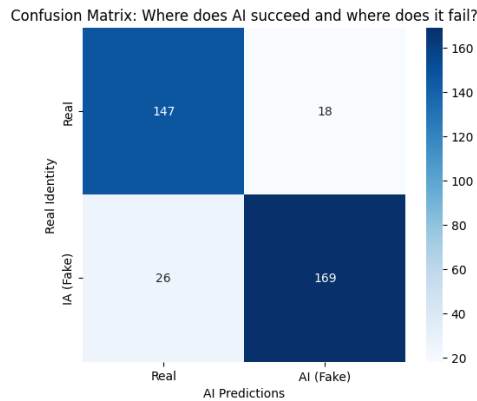


Figure 1: Confusion matrix obtained on the validation set for the Real/Fake classification task.

The confusion matrix provides further insight into the detector behaviour. Out of 165 real images, 147 were correctly classified while 18 were incorrectly identified as AI-generated. Similarly, among the 195 AI-generated images, 169 were correctly detected while 26 were classified as real.

These results indicate a balanced performance across both classes. The model achieves slightly higher precision when identifying AI-generated images (88%), meaning that when the detector predicts an image as fake, it is usually correct. On the other hand, the recall for real images reaches 88%, showing that genuine photographs are successfully recognized in most cases.

Overall, the obtained accuracy demonstrates that the proposed multi-task framework is capable of learning meaningful forensic representations despite the challenging conditions introduced by internet transmission and re-digitization processes.

7 Discussion

The experimental results demonstrate that a shared multi-task architecture can simultaneously learn information related to image authenticity and post-processing transformations.

The use of a pre-trained ResNet18 backbone significantly reduced training complexity while providing robust feature extraction capabilities. The balanced dataset construction also prevented strong class imbalance effects.

However, several limitations remain. Due to hardware constraints, only a subset of the RRDataset was used. Furthermore, the experiments were conducted using a single loss weighting configuration ($\lambda_{class} = 0.5$, $\lambda_{trans} = 0.5$). Therefore, a complete ablation study exploring different weight combinations could not be fully performed.

Future work could investigate alternative backbone architectures such as EfficientNet or Vision Transformers and evaluate additional loss-weighting strategies.

8 Conclusions

This project presented a multi-task deep learning framework for jointly detecting AI-generated images and identifying post-processing transformations under real-world conditions.

Using a balanced subset of the RRDataset, a ResNet18-based architecture with two independent classification heads was successfully implemented and trained. The proposed model produces both predictions simultaneously while sharing a common feature representation.

The results indicate that multi-task learning constitutes a viable approach for image forensics applications, allowing authenticity detection and transformation recognition to be addressed within a unified framework.

Further experimentation with larger subsets of the dataset and different loss-weighting strategies would provide additional insight into the interaction between both forensic tasks.